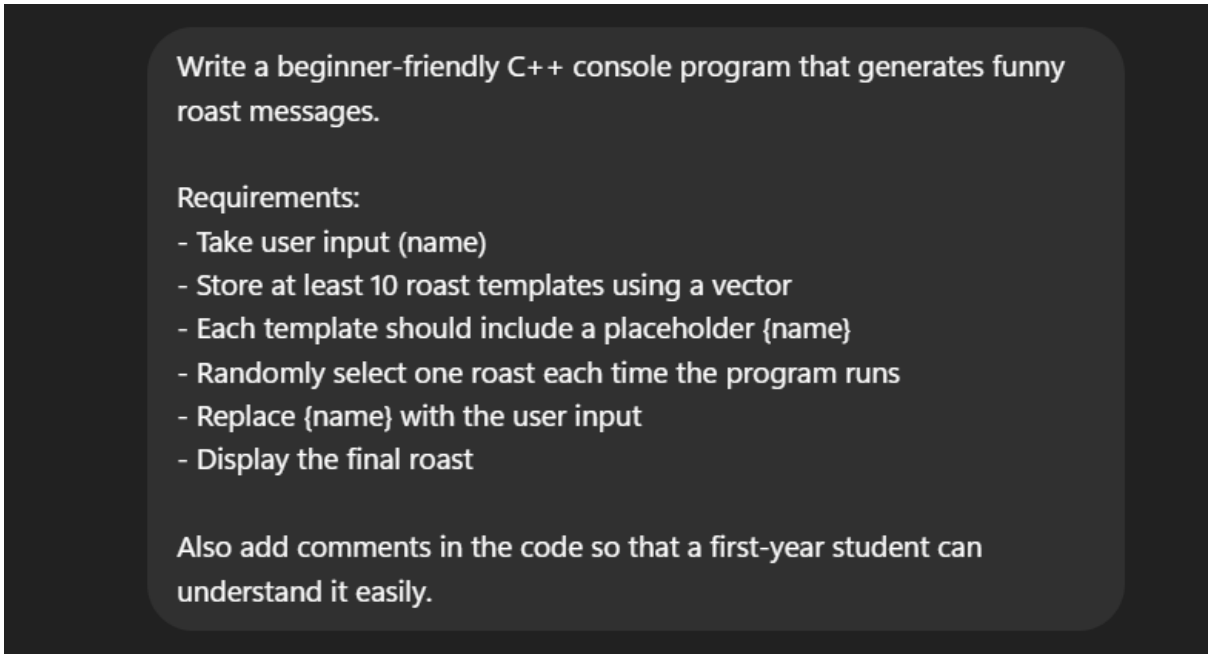
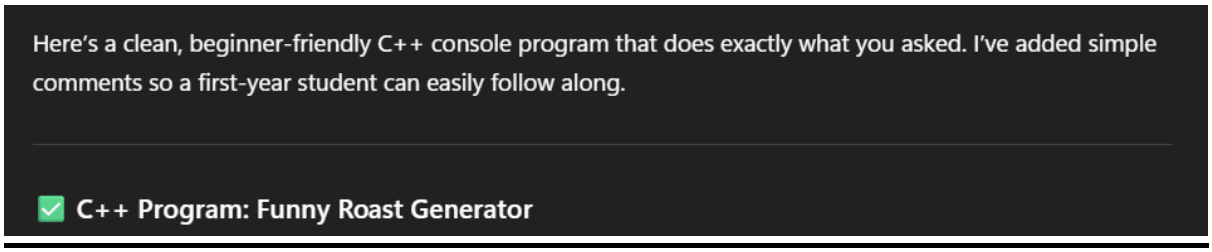
PROBLEM – 7 (AI ROAST GENERATOR)

1.Basic code prompt :



Response :



✅ C++ Program: Funny Roast Generator

C++

```cpp
#include <iostream>     // For input and output
#include <vector>       // For using vector (dynamic array)
#include <string>       // For string operations
#include <cstdlib>      // For rand() and srand()
#include <ctime>        // For time()

using namespace std;

int main() {
    // Step 1: Take user input (name)
    string name;
    cout << "Enter your name: ";
    getline(cin, name);

    // Step 2: Store roast templates in a vector
    vector<string> roasts = {
        "{name}, you're not stupid... you just have bad luck thinking.",
        "{name}, if laziness were a subject, you'd top the class.",
        "{name}, even Wi-Fi signals avoid you.",
        "{name}, your brain has too many tabs open... and none are loading.",
        "{name}, you bring everyone so much joy... when you leave the room.",
        "{name}, you're like a cloud. When you disappear, it's a beautiful day.",
        "{name}, even Google can't understand your logic.",
        "{name}, your secrets are always safe… because no one listens to you.",
        "{name}, you have something on your chin… no, the third one down.",
        "{name}, if overthinking burned calor you'd be invisible."
    };

    // Step 3: Initialize random seed (so output changes every run)
    srand(time(0));

    // Step 4: Generate random index
    int randomIndex = rand() % roasts.size();

    // Step 5: Get selected roast
    string selectedRoast = roasts[randomIndex];

    // Step 6: Replace {name} with user input
    size_t pos = selectedRoast.find("{name}");

    // Loop to replace all occurrences of {name}
    while (pos != string::npos) {
        selectedRoast.replace(pos, 6, name); // 6 is length of "{name}"
        pos = selectedRoast.find("{name}");
    }

    // Step 7: Display final roast
    cout << "\n🔥 Your Roast:\n";
    cout << selectedRoast << endl;

    return 0;
}
```

2.Improve logic prompt :

Prompt 2 (Improve Logic)
Improve the previous C++ program by making it more user-friendly.

Add:
- Proper output formatting (like headings and spacing)
- Clear prompts for user input
- Ensure the program gives a different roast each time it runs

Response :

Nice—this is exactly how you level up a basic program into something more polished 👍
Here's an improved, beginner-friendly version with better formatting, clear prompts, and stronger randomness.

✅ **Improved C++ Roast Generator (User-Friendly Version)**

C++

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <cstdlib>
#include <ctime>

using namespace std;

int main() {

    // Decorative heading for better output formatting
    cout << "====================================\n";
    cout << "       🔥 FUNNY ROAST GENERATOR 🔥      \n";
    cout << "====================================\n\n";

    // Clear input prompt
    string name;
    cout << "👉 Enter your name: ";
    getline(cin, name);

    // Check if user entered empty name
    if (name.empty()) {
        cout << "\n⚠️  Name cannot be empty. Please run again.\n";
        return 0;
    }

    // Roast templates
    vector<string> roasts = {
        "{name}, you're not stupid... you just have bad luck thinking.",
```

```
"{name}, you're not stupid... you just have bad luck thinking.",
"{name}, if laziness were a subject, you'd top the class.",
"{name}, even Wi-Fi signals avoid you.",
"{name}, your brain has too many tabs open... and none are loading.",
"{name}, you bring everyone joy... when you leave the room.",
"{name}, you're like a cloud. When you disappear, it's a beautiful day.",
"{name}, even Google can't understand your logic.",
"{name}, your secrets are safe… because no one listens to you.",
"{name}, you have something on your chin… no, the third one down.",
"{name}, if overthinking burned calories, you'd be invisible."
```

```
    // Better random seed (ensures different result every run)
    srand(time(0));

    // Generate random index
    int randomIndex = rand() % roasts.size();

    // Pick roast
    string selectedRoast = roasts[randomIndex];

    // Replace {name} with actual name
    size_t pos = selectedRoast.find("{name}");
    while (pos != string::npos) {
        selectedRoast.replace(pos, 6, name);
        pos = selectedRoast.find("{name}");
    }

    // Output formatting
    cout << "\n-------------------------------------\n";
    cout << "😈 Your Roast is Ready:\n";
    cout << "-------------------------------------\n\n";

    cout << "💬 " << selectedRoast << "\n\n";

    cout << "=====================================\n";
    cout << "   Thanks for using the program! 😄   \n";
    cout << "=====================================\n";

    return 0;
}
```

What's Improved?

- ✓ Clean heading & separators
- ✓ Clear input prompt (`👉 Enter your name`)
- ✓ Handles empty input (basic validation)
- ✓ Better user experience with emojis & spacing
- ✓ Still uses `srand(time(0))` to ensure different roast each run

3.Introduce bug situation:

I ran the program and noticed a problem.

Sometimes the placeholder {name} is not fully replaced in the roast message.

Fix the code so that all occurrences of {name} are replaced correctly, even if it appears multiple times in a sentence.
Explain what was wrong and how you fixed it.

Response:

Good catch — this is actually a common beginner mistake in string handling

❗ What was going wrong?

In your previous code, this part was the issue:

C++

```
pos = selectedRoast.find("{name}");
```

After replacing `{name}`, the program **starts searching again from the beginning** of the string.

This can cause problems when:

- `{name}` appears multiple times
- Or replacements shift the string positions

👉 Result: Sometimes one `{name}` gets skipped or not replaced properly.

✅ Fix: Move forward after each replacement

We fix it by continuing the search **after the replaced position**, not from the start.

✔ Corrected Code Snippet

Replace your old loop with this:

C++

```cpp
size_t pos = selectedRoast.find("{name}");

while (pos != string::npos) {
    selectedRoast.replace(pos, 6, name);

    // Move forward after replacement to avoid re-checking same part
    pos = selectedRoast.find("{name}", pos + name.length());
}
```

💡 **Why this works**

- `pos + name.length()` moves the search forward
- Prevents infinite loops or skipping issues
- Ensures **ALL occurrences** of `{name}` are replaced

4. Edge case handling prompt:

Modify the program to handle edge cases.

If the user enters an empty name, display an error message and exit safely.

Make sure the program does not crash.

Response:

You're thinking like a good programmer now—handling edge cases is exactly what teachers look for 👍

Your previous version *partially* handled empty input, but let's make it **safer and cleaner**, especially for cases like:

- User presses Enter (empty string)
- User enters only spaces `" "` (this should also be treated as empty)

✅ **Improved Version (With Proper Edge Case Handling)**

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <cstdlib>
#include <ctime>

using namespace std;

int main() {

    cout << "====================================\n";
    cout << "      🔥 FUNNY ROAST GENERATOR 🔥     \n";
    cout << "====================================\n\n";

    // Take input
    string name;
    cout << "👉 Enter your name: ";
    getline(cin, name);

    //  🔹 Edge Case Handling: Empty or only spaces
    bool isEmpty = true;
    for (char ch : name) {
        if (!isspace(ch)) {  // check if character is NOT a space
            isEmpty = false;
            break;
        }
    }

    if (isEmpty) {
```

```cpp
        cout << "\n⚠️ Error: Name cannot be empty or just spaces.\n";
        cout << "Please run the program again.\n";
        return 0;  // safely exit program
    }

    // Roast templates
    vector<string> roasts = {
        "{name}, you're not stupid... you just have bad luck thinking.",
        "{name}, if laziness were a subject, you'd top the class.",
        "{name}, even Wi-Fi signals avoid you.",
        "{name}, your brain has too many tabs open... and none are loading.",
        "{name}, you bring everyone joy... when you leave the room.",
        "{name}, you're like a cloud. When you disappear, it's a beautiful day.",
        "{name}, even Google can't understand your logic.",
        "{name}, your secrets are safe… because no one listens to you.",
        "{name}, you have something on your chin… no, the third one down.",
        "{name}, if overthinking burned calories, you'd be invisible."
    };

    // Random seed
    srand(time(0));

    // Random roast
    int randomIndex = rand() % roasts.size();
    string selectedRoast = roasts[randomIndex];

    // Replace ALL occurrences of {name}
    size_t pos = selectedRoast.find("{name}");
    while (pos != string::npos) {
```

```
        pos = selectedRoast.find("{name}", pos + name.length());
    }

    // Output
    cout << "\n---------------------------------------\n";
    cout << "😈 Your Roast:\n";
    cout << "---------------------------------------\n\n";
    cout << "💬 " << selectedRoast << "\n\n";

    cout << "=======================================\n";
    cout << "   Thanks for using the program! 😄   \n";
    cout << "=======================================\n";

    return 0;
}
```

🔍 What's Improved?

1. Handles empty input

```
C++
if (name.empty())
```

✔ Already covered basic case

2. Handles only spaces input (important!)

```
C++
isspace(ch)
```

✔ `" "` is now treated as invalid

✔ Prevents weird output like:

```
", you're not stupid..."
```

3. Safe Exit

```
C++
return 0;
```

5.Final enhancement:

Make the program slightly more creative.

- Add 2–3 more unique roast templates
- Improve the output display by adding symbols or emojis (console-friendly)
- Keep the code simple and readable for beginners

C++

```cpp
#include <iostream>
#include <vector>
#include <string>
#include <cstdlib>
#include <ctime>

using namespace std;

int main() {

    // Header Design
    cout << "=======================================\n";
    cout << "      🔥😈 FUNNY ROAST GENERATOR 😈🔥      \n";
    cout << "=======================================\n\n";

    // Input
    string name;
    cout << "👉 Enter your name: ";
    getline(cin, name);

    // Check for empty or spaces-only input
    bool isEmpty = true;
    for (char ch : name) {
        if (!isspace(ch)) {
            isEmpty = false;
            break;
        }
    }
```

```
    if (isEmpty) {
        cout << "\n⚠️  Error: Name cannot be empty or just spaces!\n";
        cout << "🚫 Please run the program again.\n";
        return 0;
    }

    // Roast templates (added new creative ones ⭐)
    vector<string> roasts = {
        "{name}, you're not stupid... you just have bad luck thinking.",
        "{name}, if laziness were a subject, you'd top the class.",
        "{name}, even Wi-Fi signals avoid you.",
        "{name}, your brain has too many tabs open... and none are loading.",
        "{name}, you bring everyone joy... when you leave the room.",
        "{name}, you're like a cloud. When you disappear, it's a beautiful day.",
        "{name}, even Google can't understand your logic.",
        "{name}, your secrets are safe… because no one listens to you.",
        "{name}, you have something on your chin… no, the third one down.",
        "{name}, if overthinking burned calories, you'd be invisible.",

        // ⭐ New creative roasts
        "{name}, your phone battery lasts longer than your attention span.",
        "{name}, you're proof that even evolution takes breaks.",
        "{name}, if brains were taxed, you'd get a full refund."
    };

    // Random setup
    srand(time(0));
    int randomIndex = rand() % roasts.size();
```

```
    string selectedRoast = roasts[randomIndex];

    // Replace ALL occurrences of {name}
    size_t pos = selectedRoast.find("{name}");
    while (pos != string::npos) {
        selectedRoast.replace(pos, 6, name);
        pos = selectedRoast.find("{name}", pos + name.length());
    }

    // Output Design
    cout << "\n----------------------------------------\n";
    cout << "🎯 Your Roast is Ready!\n";
    cout << "----------------------------------------\n\n";

    cout << "💬 " << selectedRoast << "\n\n";

    cout << "========================================\n";
    cout << "😂 Thanks for trying it out!\n";
    cout << "========================================\n";

    return 0;
}
```

✨ What's New?

🎯 1. Added Creative Roasts

- More variety (now 13 total)
- Slightly smarter/funnier lines

🎨 2. Better Console UI

- Headers with emojis 🔥 😈
- Clear sections using lines
- Output feels like a mini app

🧠 3. Still Beginner-Friendly

- Simple logic
- Easy loops
- Clean structure

💡 Small Pro Tip

If emojis don't show properly on your system (some compilers),
you can replace them with symbols like:

```
*** FUNNY ROAST GENERATOR ***
--> Your Roast:
```